5. Erste Versuche mit der Pipeline in Powershell

Wie einigen vielleicht schon aufgefallen ist, werden manche Eigenschaften als Ergebnis der cmdlets `Get-Process` in der Konsole nicht mit ihrem richtigen Namen angezeigt. Die Prozess-Eigenschaft *Workingset* heißt in der Konsole zum Beispiel *WS(K)*, und die Eigenschaft *CPU* heißt *CPU(s)*.

Powershell möchte uns das Leben einfacher machen und zeigt die Daten in der Konsole manchmal anders an, als sie wirklich gespeichert sind. Der Mechanismus, der das bewirkt, nennt sich **Extended Type System (ETS)**. Um sich die korrekten Eigenschaftennamen eines Objekts anzeigen zu lassen, kann man das Cmdlet *Select-Object* verwenden.

```
Get-Process | Select-Object -Property * -First 1
```

Der **senkrechte Strich** wird in Powershell als **Pipeline** (oder Pipe) bezeichnet, obwohl der Begriff Fließband eigentlich treffender wäre. Es bewirkt nämlich, dass jedes einzelne Objekt an das cmdlet `Select-Object` zur Weiterverarbeitung geleitet wird – ähnlich wie auf einem Fließband.

Hinweis: Unter Windows und auf einer deutschen Tastatur findet man den senkrechten Strich links, zusammen mit den Symbolen < und >. Man muss diese Taste auslösen, während man gleichzeitig ALT-GR gedrückt hält.

Mit dem oben stehenden Kommando zwingt man Powershell, alle Eigenschaften (mit dem ersten Parameter `-Property *`) von `Get-Process` anzugeben. Der zweite Parameter `-First 1` listet aber nur das erste zurückgegebene Objekt auf.

`Select-Object` wird eigentlich dafür verwendet, Objekteigenschaften zu filtern. Dafür gibt man als Argument für den Parameter `-Property` alle Eigenschaften an, die man benötigt.

```
Get-Process | Select-Object -Property Name, CPU, WS
```

Die erste Aufgabe zur Pipeline

Wenn man als Parameter `-Property *` eingibt, dann werden alle Eigenschaften in die Konsole geschrieben – mit ihren richtigen Namen! So kannst man ganz einfach herausfinden, wie die Eigenschaften wirklich heißen, sollte Powershell sie in der Ausgabe umbenannt haben (siehe Einleitung des Kapitels).

Oft werden dabei viel mehr Eigenschaften anzeigt, als man beim direkten Aufruf des Kommandos sieht. Das ist der funktionalen Teilung zwischen dem Betrieb an der Kommandozeile und Skripten geschuldet. Die meisten Eigenschaften braucht man nämlich eher selten und in der Konsole ist die Ausgabe dann kompakter und übersichtlicher.

Gleich kommt die erste Aufgabe. Wie bei den anderen Teilen des Tutorials gilt auch hier: Um die Übungen nachvollziehen zu können, wird Windows ab Windows 10 benötigt. Wer mit Linux oder MacOS arbeitet, kann die Microsoft-Powershell ebenfalls verwenden, sie ist plattformunabhängig. Allerdings werden einige Beispiele nicht funktionieren, da sie Windows-Funktionalitäten verwenden.

Die Aufgabe:

- **Ü5.1.1** Finde heraus, in welchen Eigenschaft sich das Erstelldatum und die Größe von Dateien und Ordnern befindet.
  Verwende das cmdlet `Get-ChildItem` in Kombination mit dem cmdlet `Select-Object` um im Verzeichnis `C:\Windows` die erste Datei mit allen Eigenschaften anzuzeigen. Suche dann die Eigenschaften für das Erstelldatum und die Länge dieser Datei.

```
Get-ChildItem /Library/ | Sort-Object -Property CreationTime | Select-Object -First 1 -Property *
```

- **Ü5.1.2** Die Dateigröße (wie alle Größenangabe) wird im Objekt in Bytes gespeichert. Wie kannst du die Größe in MB umrechnen?

```
* 1024 * 1024
```

Das folgende Script zeigt eine berechnete Spalte (= Eigenschaft) mit einer Hashtable. Der Eintrag `expression` enthält einen Script-Block bzw. ein Script im Script, wo die Berechnung durchführt wird.

```
$groesse = @{
    Name = "Groesse(kB)"
    expression = { $_.Length/1kB }
}

Get-ChildItem -Path C:\Windows -File | Select-Object -Property name, $groesse
```

- **Ü5.1.3** Mit `Get-Process` kannst du dir die laufenden Programme anzeigen lassen – in welchen Eigenschaften befinden sich der Herstellername und der Pfad zur Datei?

```
path
```

- **Ü5.1.4** `Invoke-WebRequest` ruft Inhalte von einer Webseite im Internet ab. Lass dir von der Webseite `www.example.com` den Statuswert, die Beschreibung des Statuswertes, das Encoding und den HTML-Inhalt ausgeben.

```

```

- **Ü5.1.5** Was für einen Datentyp bekommst du zurückgeliefert, wenn du ein Objekt mit `Select-Object` gefiltert hast?

```

```

Daten in der Pipeline sortieren

Manchmal kommen die Objekte nicht in der Reihenfolge zurück, in der man sie benötigt. Mit `Sort-Object` kann man die Sortierreihenfolge in der Pipeline anpassen. Man muss nur sagen, nach welcher Eigenschaft es sortieren soll, indem man den Eigenschaft-Namen im Parameter `-Property` angibt.

```
Get-ChildItem -Path C:\Windows | Sort-Object -Property CreationTime
# Die Eigenschaft CreationTime wird nicht angezeigt. Für die Anzeige von LastWriteTime muss
zusätzlich Select-Object verwendet werden. Das Ergebnis ist aufsteigend sortiert
Get-ChildItem -Path C:\Windows | Sort-Object -Property CreationTime | Select-Object -
Property Creation, Name
# Mit dem Parameter -Descending wird absteigend sortiert
Get-ChildItem -Path C:\Windows | Sort-Object -Property CreationTime -Descending
```

Man kann in der Pipeline auch mehrere Cmdlets verknüpfen – dazu muss man nur ein weiteres Pipe-Symbol an das Ende der Pipeline stellen und dann das nächste Kommando anhängen.

Dazu hier die nächsten Aufgaben:

- **Ü5.2.1** Sortiere die Dateien unter `C:\Windows\System32` der Größe nach.
  Hinweis: Die Größe entspricht der Länge der Datei.

  ```
  Get-ChildItem /Library/ I Sort-Object -Property size
  ```

- **Ü5.2.2** Sortiere die Dateien im Windows-Ordner nach dem Datum der letzten Änderung, aber absteigend!
  Hinweis: Wenn du nicht weiter weißt – die Hilfe weiß, wie es geht 😉

  ```
  Get-ChildItem /Library/ I Sort-Object -Property LastWriteTime -Descending
  ```

- **Ü5.2.3** Sortiere die laufenden Programme nach Hersteller-Namen.
  Hinweis: Die laufenden Programme werden mit `Get-Process` angezeigt.

Daten filtern

In der PowerShell kann man oft direkt mit den Cmdlets filtern, z.B. `Get-EventLog -LogName Application -EntryType Error`, wobei hier `-EntryType Error` ein Filter ist. Es nur die Error-Einträge ermittelt. Wenn das Cmdlet keinen passenden Filter anbieten, dann kann `Where-Object` verwendet werden.

`Where-Object` ist ein äußerst mächtiges Cmdlet. `Where-Object` erlaubt es, Objekte, die man nicht verarbeiten möchte, aus der Pipeline zu entfernen.

Damit `Where-Object` weiß, welche Objekte entfernt werden sollen, muss man einen Filter übergeben. `Where-Object` unterscheidet dabei nur zwischen `wahr` und `falsch`.

Welche Objekte gewünscht sind, muss Powershell über einen Boolschen Filter übergeben werden. Der Filter enthält die zu untersuchende Eigenschaft, den Wert, mit dem die Eigenschaft verglichen werden soll, und den Vergleichsoperator, der das Verhältnis von Eigenschaft und Wert angibt.

Dazu werden sogenannte Vergleichsoperatoren verwendet, allerdings in textueller Darstellung. Die folgende Tabelle zeigt die möglichen Operatoren und ihre Bedeutung:

Vergleichsoperator	Mathematischer / SQL-Operator	Bedeutung
-eq	==	equal
-neq	< > oder !=	not equal
-lt	<	less than
-gt	>	greater than
-le	<=	less or equal
-ge	>=	greater or equal
-like	Like (in SQL)	wie equal mit Platzhaltern (*,?)
-notlike	Notlike (in SQL)	wie not equal mit *,?
-match	-	wie like mit komplexen Mustern
-contains	In (SQL)	Suchbegriff in Array?
-notcontains	Notin (SQL)	Suchbegriff nicht in Array?
-in	In (SQL)	Array enthält Suchbegriff?
-notin	Notin (SQL)	Array enthält Suchbegriff nicht?
-not / !	! / Not (SQL)	Negierung, dreht Ergebnis um

Die erste Spalte in der Tabelle stellt den Vergleichsoperator dar, wie wir ihn in Powershell eingeben müssen, die zweite Spalte zeigt den mathematischen Operator beziehungsweise den Vergleichsoperator aus SQL (3. Klasse INSY) an.

Powershell verwendet nicht direkt mathematische Operatoren, weil es Überschneidungen mit in Powershell anderweitig verwendeten Operatoren wie dem `=` gibt, das ja bereits für die Variablenzuweisung verwendet wird – die Operatoren hat das Powershell-Team hier übrigens aus der **Linux-Kommandozeile** übernommen.

Ein Anwendungsfall für `Where-Object` ist zum Beispiel das Anzeigen aller Dienste, die aktuell nicht ausgeführt werden. Im Dienst-Objekt kann man das der Eigenschaft `Status` entnehmen. Das Cmdlet `Get-Service` kann den Dienststatus nicht filtern, aber mit `Where-Object` ist das kein Problem:

```
Get-Service | Where-Object -Property Status -EQ -Value "Stopped"
```

`Where-Object` verwendet hier drei Parameter: `-Property` gibt die zu untersuchende Eigenschaft an, `-EQ` steht für equal und entspricht dem Gleichheitszeichen und `-Value` gibt den Wert an, der in der Spalte Status stehen muss, damit `Where-Object` das Objekt nicht aus der Pipeline entfernt.

Damit kommen wir zu den letzten Aufgaben aus diesem Block:

- **Ü5.3.1** Liste alle Dateien innerhalb des Windows\System32-Ordners auf, die größer als 1 MB sind. Tipp: Powershell kann Größenangaben automatisch umrechnen, indem du die Zahl direkt gefolgt von der Einheit angibst. 10 KB wird von Powershell also als 10240 interpretiert. Probiere es aus. Gib in der

Konsole 10 MB ein – Powershell gibt dir die Größe in Bytes zurück.

- **Ü5.3.2** Liste alle Dateien aus dem Windows\System32-Ordner auf, deren Erstelldatum vor dem 1.1.2021 liegt. Verwende ein `DateTime`-Objekt als Vergleichswert – das Erstelldatum ist ebenfalls als `DateTime` gespeichert, und `DateTime`-Werte kann man direkt miteinander vergleichen.

- **Ü5.3.3** Liste alle Prozesse auf, die mehr als 10 MB Arbeitsspeicher benötigen. Der Arbeitsspeicher wird in der Eigenschaft WS oder Workingset (zwei Eigenschaften mit identischem Inhalt) angegeben.

Die Größe eines Ordners ermitteln

Mit der Pipeline steht ein mächtiges Werkzeug zur Verfügung, um große Mengen an Daten mit wenigen Zeilen Code zu verarbeiten. Ein sehr hilfreiches Cmdlet ist `Measure-Object`. Es bildet **Summen**, **Durchschnittswerte**, **Maxima**, **Minima** und zählt nebenbei noch die **Anzahl** der Objekte. Man setzt es am **Ende der Pipeline** ein. Will man zum Beispiel wissen, wie viele Cmdlets Powershell zur Verfügung stellt, gibt man folgenden Befehl ein:

```
Get-Command | Measure-Object
```

Um Summen zu bilden, muss man allerdings noch zwei weitere Parameter anfügen. Der eine ist die Eigenschaft, über die die Summe gebildet werden soll.

Den anderen finde selbst heraus und versuche, die folgende Übung zu lösen:

- **Ü5.4.1** Ermittle die Anzahl aller Dateien im Windows-Ordner.

- **Ü5.4.2** Wie viele Powershell-Verben (die im Cmdlet-Namen verwendet werden dürfen) gibt es?

- **Ü5.4.2** Ermittle die Größe des Windows-Ordners, indem du die Summe über die Größe aller Dateien bildest.

```
Get-ChildItem "/System" -Recurse -File | Measure-Object -Property Length -Sum
```

- **Ü5.4.2** Wie viel Arbeitsspeicher benötigt der größte, gerade laufenden Windows-Prozess? Den verwendeten Arbeitsspeicher findest du in der Eigenschaft `WorkingSet` der Prozess-Objekte.

```
Get-Process | Sort-Object -Property WorkingSet -Descending | Select-Object -First 1 -Property *
```